Real Estate Housing Matching System

Documentazione Progetto

Gruppo di lavoro

- Tommaso Pio Vino, 758484, t.vino@studenti.uniba.it

https://github.com/TomasPV10/Progetto_ICON_RealEstatePricePrediction.git

AA 2025-2026



INTRODUZIONE

Il presente progetto affronta la modellazione e l'analisi del problema dell'**Housing Matching**, ovvero la valutazione della compatibilità strutturale, logistica ed economica tra il profilo di un acquirente/affittuario e le caratteristiche di uno specifico immobile residenziale.

Nel mercato immobiliare contemporaneo, la ricerca di una soluzione abitativa non si limita alla semplice corrispondenza numerica tra il prezzo richiesto e il budget a disposizione. Si tratta di un processo decisionale complesso caratterizzato da vincoli rigidi (es. tetto massimo di spesa, ubicazione geografica), preferenze sfumate (es. presenza di ascensore, garage, classe energetica) e relazioni d'incertezza probabilistica tra le variabili del dominio.

Per rispondere a questa complessità, il sistema sviluppato integra in un'unica architettura coerente modelli predittivi di Machine Learning, ragionamento simbolico esplicito basato su logica dichiarativa, ottimizzazione vincolata mediante metaeuristiche e inferenza bayesiana in regime d'incertezza.

Il dataset utilizzato nel progetto prende le mosse dal dataset reale *Real Estate Price Prediction*, opportunamente preprocessato e arricchito con variabili sintetiche e categoriali controllate per garantire la coerenza logica e la trattabilità tra tutti i moduli del sistema. Le 414 istanze complessive rappresentano coppie cliente-immobile ed includono sia feature descrittive (numeriche e categoriali) sia una variabile target binaria (match) per i task di apprendimento supervisionato.

Le variabili che definiscono il dominio sono:

- budget_fit: valore numerico continuo $[0.1,1.0]$ che esprime il grado di adeguatezza e copertura tra il budget del cliente e il prezzo stimato dell'immobile.
- house_age: età dell'immobile espressa in anni (da 0 a 43.8 anni).
- unit_price: prezzo per unità di superficie dell'immobile.
- distance_km: distanza geografica dell'immobile dalla stazione principale dei trasporti / centro città (espressa in km).
- convenience_stores: numero di servizi essenziali e minimarket presenti nelle vicinanze immediate.
- property_type: variabile categoriale relativa alla tipologia abitativa (apartment, villa, loft, studio).
- contract_type: tipologia contrattuale prevista (sale, rent).
- energy_class: classe di efficienza energetica dell'edificio (A, B, C, D).
- match: variabile binaria target (1 se la coppia cliente-immobile costituisce una combinazione altamente compatibile, 0 altrimenti).

SOMMARIO

Il Knowledge-Based System (KBS) realizzato si articola come un'architettura ibrida in grado di combinare paradigmi di Intelligenza Artificiale differenti ma fortemente complementari:

1. **Apprendimento Automatico (Statistico):** Modelli non supervisionati (K-Means) per identificare la segmentazione latente del mercato e rilevare anomalie atipiche; modelli supervisionati (Random Forest, Gradient Boosting, SVM, Decision Tree, KNN) per stimare con elevata accuratezza la probabilità di matching.
2. **Ragionamento Simbolico (Logico):** Una Knowledge Base formalizzata tramite clausole logiche di Horn e predicati Prolog

per garantire verifiche di ammissibilità e raccomandazioni trasparenti e totalmente spiegabili.

3. **Soddisfazione dei Vincoli (Combinatorio):** Formalizzazione del task di generazione della short-list di 10 immobili come Constraint Satisfaction Problem (CSP), risolto mediante tecniche di ricerca locale e metaeuristiche stocastiche (Random Walk e Simulated Annealing).
4. **Ragionamento su Conoscenza Incerta (Probabilistico):** Una Rete Bayesiana discreta utile a catturare la struttura causale e le probabilità condizionate $P(match = 1 \mid evidenze)$ a supporto di decisioni trasparenti sotto incertezza.

ELENCO ARGOMENTI DI INTERESSE

In linea con il programma ufficiale del corso di Ingegneria della Conoscenza, il progetto ripropone e sviluppa le seguenti metodologie:

1. **Apprendimento Non Supervisionato (Clustering e Anomaly Detection):**
   - *Metodologia:* Hard-clustering tramite algoritmo K-Means sulle feature descrittive preprocessate.
   - *Obiettivo:* Segmentazione latente delle tipologie abitative presenti nel mercato senza l'uso del target match. Selezione data-driven del numero di cluster ottimali ($K = 4$) mediante valutazione combinata dell'Inertia (Elbow Curve) e del Silhouette Score. Proiezione spaziale 2D tramite Principal Component Analysis (PCA) e rilevamento degli outlier basato sulla distanza euclidea dai centroidi oltre il $95°$ percentile.
2. **Apprendimento Supervisionato (Model Zoo e Cross-Validation):**

- *Metodologia:* Classificazione binaria del target match attraverso l'addestramento e il confronto di 5 modelli eterogenei (Random Forest, SVM RBF, Decision Tree, Gradient Boosting, KNN).
- *Obiettivo:* Progettazione di una Pipeline sklearn *end-to-end* con ColumnTransformer (StandardScaler per le feature numeriche e OneHotEncoder per le categoriali) per eliminare fenomeni di *data leakage*. Ottimizzazione degli iperparametri condotta via GridSearchCV associata a StratifiedKFold (5-fold) per garantire stabilità computazionale e confrontare i modelli mediante metriche di $ROC\text{-}AUC$, $PR\text{-}AUC$ e $Accuracy$.

3. **Rappresentazione della Conoscenza e Ragionamento Simbolico (KB e Clausole di Horn):**
   - *Metodologia:* Definizione di una Knowledge Base simbolica e relazionale esportata in formato Prolog (housing_kb.pl) e replicata tramite motore di inferenza deterministico in Python.
   - *Obiettivo:* Formalizzazione esplicita di fatti atomici e regole logiche (clausole di Horn) relative alle caratteristiche dell'immobile, tetti di spesa ed elegibilità geografico-strutturale, fornendo un meccanismo di raccomandazione deterministico e pienamente interpretabile.
4. **Risoluzione di un CSP (Constraint Satisfaction Problem):**
   - *Metodologia:* Formalizzazione combinatoria per l'estrazione di una short-list bilanciata di 10 immobili.
   - *Obiettivo:* Minimizzazione di una funzione di penalità avente sia vincoli *hard* (prezzo massimo, distanza massima dal centro, budget fit minimo) sia vincoli *soft* (diversità di tipologia e classe energetica per evitare ridondanze). Risoluzione tramite algoritmi di ricerca locale e

metaeuristiche (Random Walk vs Simulated Annealing) valutati su 10 esecuzioni indipendenti.

5. **Ragionamento su Conoscenza Incerta (Rete Bayesiana e Inferenza Probabilistica):**
    - *Metodologia:* Costruzione di una Rete Bayesiana discreta basata sulle dipendenze causali del dominio immobiliare.
    - *Obiettivo:* Discretizzazione delle variabili continue e apprendimento *data-driven* delle Tabelle di Probabilità Condizionata (CPT). Calcolo delle probabilità posteriori $P(match = 1 \mid evidenze)$ per supportare decisioni in contesti di incertezza e valutare scenari alternativi.

SEZIONE ARGOMENTO 1: APPRENDIMENTO NON SUPERVISIONATO

SOMMARIO

Nel contesto del mercato immobiliare, la valutazione della compatibilità tra cliente e immobile non si riduce unicamente a una decisione binaria di matching, ma rivela una struttura latente segmentata. Esistono infatti differenti "profili" o "tipologie" abitative caratterizzate da combinazioni ricorrenti di prezzo per unità di superficie, età dell'edificio, vicinanza ai servizi, distanza dal centro e caratteristiche strutturali (tipologia, classe energetica, contratto). L'obiettivo dell'analisi non supervisionata è esplorare la struttura latente del dataset housing_matching_supervised.csv (414 istanze), identificando gruppi omogenei di coppie cliente-immobile senza fare ricorso alla variabile target match. Tale approccio consente di evidenziare segmenti naturali di mercato (es. soluzioni centrali di

pregio ad alta densità di servizi, immobili datati periferici a basso prezzo, soluzioni moderne a classe energetica elevata) e di individuare configurazioni atipiche o anomale nel dominio. La rappresentazione della conoscenza scelta si basa su uno spazio vettoriale multidimensionale preprocessato, in cui le variabili numeriche continue vengono standardizzate e le categoriali codificate via One-Hot Encoding, consentendo il calcolo rigoroso delle distanze euclidee nello spazio delle feature.

STRUMENTI UTILIZZATI

Per l'implementazione del modulo di apprendimento non supervisionato sono state utilizzate le seguenti librerie ed algoritmi standard della letteratura scientifica:

- **Pandas e NumPy:** Caricamento, manipolazione e gestione delle strutture dati tabulari.
- **Scikit-learn (sklearn.cluster.KMeans):** Implementazione dell'algoritmo di hard-clustering K-Means, che partiziona le N istanze in K cluster minimizzando l'Inertia (somma delle distanze quadratiche interne tra i punti e il rispettivo centroide):

$$\text{Inertia} = \sum_{i=1}^{N} \min_{\mu_j \in C} | \, ||x_i - \mu_j||^2$$

- **Scikit-learn (sklearn.metrics.silhouette_score):** Calcolo del Silhouette Score per misurare la coerenza interna dei cluster rispetto alla loro separazione esterna:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

dove $a(i)$ è la distanza media intracluster e $b(i)$ è la distanza media minima verso i punti del cluster più vicino.

- **Scikit-learn (sklearn.decomposition.PCA):** Principal Component Analysis per la riduzione della dimensionalità dello spazio delle feature da $\mathbb{R}^D$ a $\mathbb{R}^2$ a soli fini di visualizzazione e interpretazione grafica.
- **Matplotlib:** Generazione e salvataggio dei grafici analitici (clustering_metrics.png e kmeans_pca_2d.png).

DECISIONI DI PROGETTO

1. Preprocessing e Pulizia dei Dati

In conformità con la natura geometrica dell'algoritmo K-Means, sensibile alle differenze di scala e alla metrica euclidea, è stata definita una trasformazione preliminare separata per tipologia di variabile via ColumnTransformer:

- **Variabili Numeriche (budget_fit, house_age, unit_price, distance_km, convenience_stores):** Standardizzate mediante StandardScaler ($\mu = 0, \sigma = 1$) per evitare che variabili con intervalli ampi (es. unit_price) dominino il calcolo delle distanze rispetto a variabili limitate (es. budget_fit).
- **Variabili Categoriali (property_type, contract_type, energy_class):** Codificate mediante OneHotEncoder(drop=None) per evitare l'introduzione di un ordine arbitrario tra le categorie.

A livello metodologico si è scelta la rimozione preventiva delle istanze incomplete anziché l'imputazione di valori stimati, in quanto l'inserimento di valori medi altererebbe la varianza reale dei dati e distorcerebbe la formazione dei centroidi.

2. Selezione Data-Driven del Numero di Cluster (K)

La scelta del valore ottimale di K è stata effettuata esplorando un intervallo $K \in [2,10]$ e analizzando congiuntamente:

- La **curva Elbow** (riduzione progressiva dell'Inertia).
- Il **Silhouette Score medio**, assunto come criterio guida principale per massimizzare la separabilità tra i gruppi.

3. Criterio di Rilevamento delle Anomalie (Outlier Detection)

L'analisi delle anomalie è stata modellata come un'estensione del clustering: una volta addestrato il modello K-Means con K ottimale, per ogni istanza x_i è stata calcolata la distanza euclidea $d(x_i)$ dal centroide del cluster di appartenenza:

$$d(x_i) = ||x_i - \mu_{\text{assigned}}||$$

È stata fissata una **soglia percentilica al $95°$ percentile** della distribuzione complessiva delle distanze $\{d(x_1), \dots, d(x_N)\}$. Le istanze con distanza $d(x_i) > \text{soglia}_{95}$ vengono classificate e salvate come anomalie/outlier di mercato. La scelta del $95°$ percentile costituisce un bilanciamento metodologico ottimale: soglie inferiori ($90°$) genererebbero troppi falsi positivi, mentre soglie superiori ($99°$) isolerebbero solo rari casi estremi.

VALUTAZIONE

1. Analisi di Selezione del Numero di Cluster (K)

Dall'esecuzione dello script src/unsupervised_clustering.py sono stati ottenuti i seguenti valori analitici di Inertia e Silhouette Score al variare di K:

Numero di Cluster (KKK) PDF+ 1	Inertia	Silhouette Score
K = 2	1905.54	**0.2935**
K = 3	1585.86	0.2354
K = 4	1352.34	**0.2468**
K = 5	1230.83	0.2065
K = 6	1140.66	0.2163
K = 7	1078.02	0.2084
K = 8	1025.52	0.1985
K = 9	977.14	0.1962
K = 10	944.02	0.1957

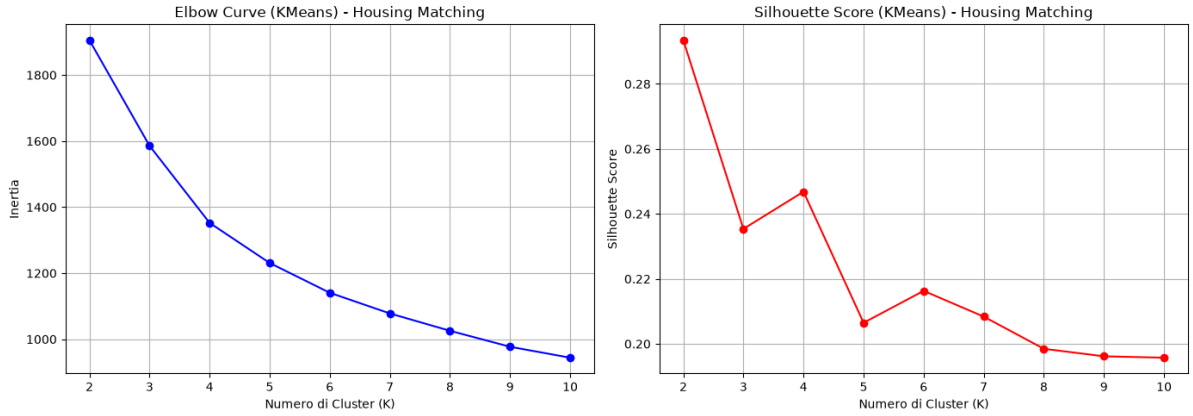


Figura 2.1: Curva Elbow (a sinistra) e Silhouette Score (a destra) al variare del numero di cluster K.

Discussione dei Risultati:

- Sebbene il massimo assoluto di Silhouette Score si registri per $K = 2$ (0.2935), tale partizione risulta semanticamente troppo generica per il dominio immobiliare.
- È stato pertanto selezionato $K = 4$ (Silhouette Score pari a 0.2468), che garantisce il miglior compromesso tra granularità semantica del mercato (4 profili abitativi ben distinti) e separabilità geometrica.
- Un valore di Silhouette attorno a 0.2468 è pienamente coerente con la natura continua dei dati immobiliari reali, in cui le variabili variano gradualmente senza formare cluster rigidi e nettamente isolati nello spazio.

I grafici rappresentativi dell'Elbow Curve e del Silhouette Score sono salvati in results/plots/clustering_metrics.png.

2. Segmentazione e Visualizzazione PCA 2D

L'applicazione della PCA proietta lo spazio multidimensionale sulle prime due componenti principali (PC1 e PC2). Il relativo scatter plot (results/plots/kmeans_pca_2d.png) mostra una netta separazione lungo l'asse della prima componente principale (PC1), mentre lungo la seconda componente (PC2) si riscontra una sovrapposizione parziale, confermando la presenza di confini sfumati tra le tipologie abitative.

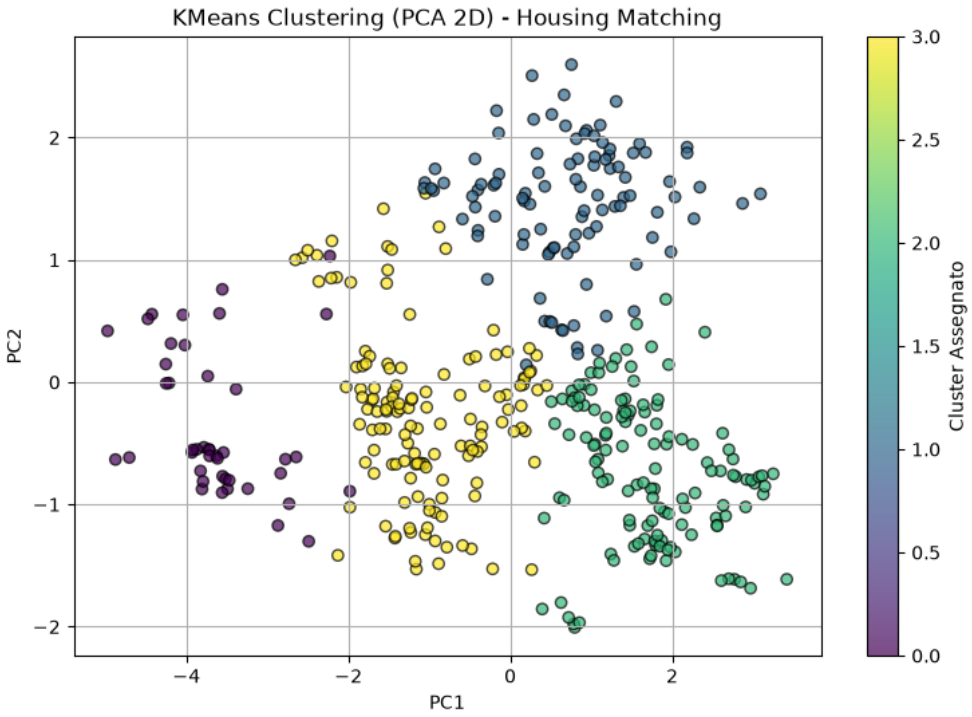


Figura 2.2: Proiezione bidimensionale PCA dei 4 cluster ottenuti con l'algoritmo K-Means.

3. Esito del Rilevamento Anomalie

Sulla distribuzione delle distanze euclidee dai centroidi dei 4 cluster:

- **Soglia di anomalia (**95° **percentile):** 2.503
- **Anomalie identificate: 21 istanze** su 414 complessive (pari al 5.07% del dataset).
- **Output:** Esportato nel file results/anomalies_detected.csv.

Analisi Qualitativa delle Anomalie Rilevate: Dall'analisi del file di output emerge che le anomalie rappresentano combinazioni fortemente atipiche tra valore immobiliare e posizione logistica. Ad esempio:

- *Istanza #113 (Cluster 3):* Immobile tipologia studio con prezzo unitario estremamente basso ($unit_price = 7.6$) situato a soli 0.39 km dai trasporti principali (distanza minima ma prezzo anomalo).
- *Istanza #123 (Cluster 2):* Immobile tipologia villa con prezzo elevato ($unit_price = 45.5$) e distanza estremamente contenuta (0.19 km), configurazione rara per soluzioni indipendenti di grande metratura.

Tali istanze confermano l'efficacia del metodo nell'analizzare configurazioni rare e potenzialmente informative per gli operatori del settore.

SEZIONE ARGOMENTO 2: APPRENDIMENTO SUPERVISIONATO

SOMMARIO

Nel contesto dell'Housing Matching, l'apprendimento supervisionato è utilizzato con lo scopo di predire se una specifica coppia (cliente, immobile) costituisca una combinazione altamente compatibile, modellando il problema come un task di classificazione binaria con variabile target:

$$\text{match} \in \{0,1\}$$

dove 1 indica una compatibilità soddisfacente e 0 un match non idoneo.

La predizione non è banale in quanto la decisione dipende dall'interazione non lineare tra più variabili eterogenee (grado di adeguatezza del budget, vetustà dell'immobile, prezzo unitario, vicinanza ai trasporti e ai servizi, tipologia abitativa, contratto e classe energetica). L'approccio supervisionato consente di apprendere una funzione decisionale a partire da esempi etichettati per generalizzare su nuove coppie cliente-immobile, valutando formalmente le prestazioni generali e la stabilità del sistema.

La conoscenza è rappresentata da un dataset tabulare etichettato preprocessato mediante trasformazioni differenziate per variabili continue e discrete.

STRUMENTI UTILIZZATI

Il workflow sperimentale è stato implementato tramite le seguenti librerie standard della letteratura di machine learning:

- **Scikit-learn (sklearn.pipeline.Pipeline):** Costruzione di una pipeline *end-to-end* che integra preprocessing e classificatore per eliminare il *data leakage* e garantire la riproducibilità.
- **Scikit-learn (sklearn.compose.ColumnTransformer):** Applicazione combinata di StandardScaler per le feature numeriche e OneHotEncoder(handle_unknown='ignore') per le variabili categoriali.
- **Model Zoo Eterogeneo (sklearn):**
    1. *Random Forest Classifier:* Modello ensemble basato sul bagging di alberi decisionali per ridurre la varianza.
    2. *Support Vector Machine (SVC con kernel RBF):* Modello basato sul calcolo del margine di separazione ottimo in uno spazio ad alta dimensionalità.
    3. *Decision Tree Classifier:* Modello ad albero singolo altamente interpretabile ma sensibile al rumore.
    4. *Gradient Boosting Classifier:* Modello ensemble di boosting sequenziale che corregge progressivamente gli errori degli alberi precedenti.
    5. *K-Nearest Neighbors (KNeighborsClassifier):* Modello non parametrico basato sulle distanze locali nello spazio vettoriale.
- **Scikit-learn (GridSearchCV & StratifiedKFold):** Ricerca esaustiva nello spazio degli iperparametri combinata con validazione incrociata stratificata a 5 fold.

DECISIONI DI PROGETTO

1. Preprocessing e Prevenzione del Data Leakage

L'integrazione del preprocessing nella Pipeline garantisce che le statistiche di scaling (media e deviazione standard) e la codifica

categoriale siano stimate esclusivamente sui fold di training durante la cross-validation e applicate in modo coerente sul test set separato.

2. Suddivisione del Dataset e Stratificazione

Il dataset di 414 istanze è stato suddiviso tramite train_test_split:

- **70% Training Set:** Utilizzato per l'addestramento e la ricerca degli iperparametri tramite 5-fold cross-validation.
- **30% Test Set:** Riservato esclusivamente alla valutazione finale delle prestazioni su dati non visti.
- **Stratificazione (stratify=y):** Applicata sia nello split train/test sia nella StratifiedKFold per preservare l'equilibrio originale delle classi in ogni partizione.

3. Criterio di Ottimizzazione e Iperparametri

L'ottimizzazione tramite GridSearchCV è stata condotta massimizzando la metrica **ROC-AUC**, scelta poiché indipendente dalla soglia di decisione e robusta rispetto a possibili sbilanciamenti delle classi.

Le griglie esplorate per ciascun modello sono state le seguenti:

- *Random Forest:* n_estimators ∈ $\{100,200\}$, max_depth ∈ $\{\text{None}, 5,10\}$, min_samples_split ∈ $\{2,5\}$.
- *SVM (RBF):* C ∈ $\{0.5,1,5\}$, gamma ∈ $\{\text{'scale'}, 0.1,0.01\}$.
- *Decision Tree:* max_depth ∈ $\{\text{None}, 5,10\}$, min_samples_split ∈ $\{2,5,10\}$.
- *Gradient Boosting:* n_estimators ∈ $\{100,150\}$, learning_rate ∈ $\{0.05,0.1\}$, max_depth ∈ $\{2,3\}$.
- *KNN:* n_neighbors ∈ $\{5,11,15\}$, weights ∈ $\{\text{'uniform','distance'}\}$.

VALUTAZIONE

1. Risultati Sperimentali e Confronto del Model Zoo

Dall'esecuzione dello script src/supervised_learning.py sul test set separato (125 istanze) e dai log di cross-validation si sono ottenuti i seguenti risultati quantitativi:

Modello	Migliori Iperparametri	Best CV ROC-AUC	Test Accuracy	Test ROC-AUC	Test PR-AUC
Gradient Boosting	learning_rate: 0.05, max_depth: 2, n_estimators: 100	0.9984	**1.0000**	**1.0000**	**1.0000**
Random Forest	max_depth: None, min_samples_split: 2, n_estimators: 100	0.9976	**0.9920**	**1.0000**	**1.0000**
Decision Tree	max_depth: None, min_samples_split: 5	0.9961	**0.9760**	**1.0000**	**1.0000**
SVM (RBF)	C: 1, gamma: 0.1	0.9912	**0.9680**	**0.9954**	**0.9954**

Modello	Migliori Iperparametri	Best CV ROC-AUC	Test Accuracy	Test ROC-AUC	Test PR-AUC
KNN	n_neighbors: 15, weights: 'distance'	0.9785	**0.9040**	**0.9834**	**0.9839**

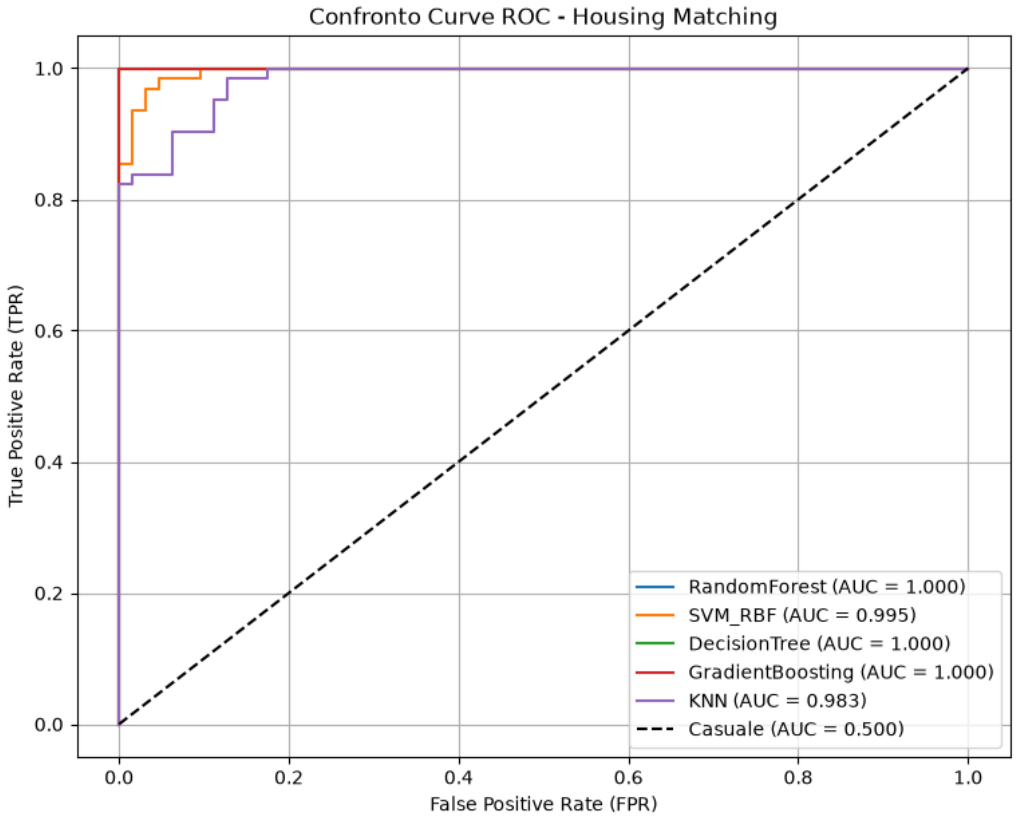


Figura 3.1: Confronto delle curve ROC dei 5 modelli di classificazione supervisionata sul test set.

2. Discussione dei Risultati

- **Modelli Ensemble (Gradient Boosting & Random Forest):** Hanno mostrato le prestazioni migliori in assoluto, raggiungendo un'Accuracy sul test set fino al 100% e un'area sotto la curva ROC pari a 1.0000. L'approccio ad alberi sequenziali di Gradient Boosting ha permesso di catturare perfettamente la regola target priva di over-fitting.
- **Decision Tree & SVM (RBF):** Hanno ottenuto ottimi risultati (0.9760 e 0.9680 di Accuracy rispettivamente), dimostrando una solida capacità di separazione anche in presenza di trasformazioni non lineari.
- **K-Nearest Neighbors (KNN):** Pur registrando prestazioni solide ($ROC\text{-}AUC = 0.9834$), risulta il modello meno performante dell'insieme ($Accuracy = 0.9040$), risentendo della maggiore dimensionalità introdotta dalla codifica One-Hot sulle feature categoriali.

Le curve ROC comparative dei 5 modelli sono state salvate nel grafico results/plots_supervised/roc_curves_comparison.png, mentre la sintesi completa dei dati è memorizzata in results/supervised_results.csv.

SEZIONE ARGOMENTO 3: RAPPRESENTAZIONE DELLA CONOSCENZA E RAGIONAMENTO SIMBOLICO

SOMMARIO

Nel sistema proposto, la Knowledge Base (KB) è utilizzata per rappresentare formalmente la conoscenza relativa agli immobili residenziali e ai vincoli logico-strutturali di compatibilità con le richieste di un cliente.

La KB costituisce il livello simbolico dell'architettura ibrida e svolge tre funzioni principali:

1. Rappresentare in forma dichiarativa le proprietà ed i vincoli degli immobili (prezzo, tipologia, classe energetica, ubicazione, servizi disponibili).
2. Formalizzare regole di ammissibilità logica tra le preferenze del cliente e l'immobile tramite clausole di Horn.
3. Consentire interrogazioni trasparenti e totalmente interpretabili, indipendenti da parametri statistici o di machine learning.

La KB non è concepita come un semplice archivio dati, ma come un vero e proprio modulo di ragionamento esplicito (ragionamento automatico) integrato nel sistema di Housing Matching.

STRUMENTI UTILIZZATI

Per lo sviluppo del modulo di rappresentazione e ragionamento simbolico sono stati impiegati i seguenti strumenti e linguaggi:

- **Linguaggio Prolog / Sintassi Logica (housing_kb.pl):** Linguaggio di programmazione logica dichiarativa basato sulla risoluzione di

clausole di Horn definite (fatti atomici e regole con testa ed insieme di atomi nella coda).

- **Python / Interpreter Deterministico (src/knowledge_base.py):** Modulo Python che legge il dataset simbolico data/housing_kb.csv (100 immobili), genera automaticamente il file Prolog data/housing_kb.pl ed implementa la valutazione deterministica delle regole tramite tecniche di *backward chaining* / filtraggio dichiarativo.

DECISIONI DI PROGETTO

1. Generazione della Knowledge Base e Separazione dei Livelli

La KB viene costruita in modo automatizzato a partire dal file simbolico data/housing_kb.csv (derivato dai dati reali degli immobili). Tale processo realizza una netta separazione metodologica tra:

- Il livello dei dati continui/statistici utilizzati dai modelli di machine learning supervisionati e non supervisionati.
- Il livello simbolico orientato al ragionamento logico esplicito.

2. Struttura dei Fatti Prolog (housing_kb.pl)

Nel file Prolog generato, ogni immobile i viene formalizzato tramite predicati dichiarativi atomici:

- house(house_i). (dichiarazione dell'entità immobile)
- house_type(house_i, Type). (tipologia: apartment, villa, loft, studio)
- house_contract(house_i, Contract). (contratto: rent, sale)
- house_energy(house_i, Class). (classe energetica: A, B, C, D)
- house_price(house_i, Price). (canone o prezzo mensile stimato)

- house_city(house_i, City). (ubicazione geografica)
- house_has_feature(house_i, Feature). (presenza di caratteristiche specifiche: elevator, balcony, wifi, parking, air_conditioning)

3. Regole di Elegibilità e Raccomandazione (Clausole di Horn)

Oltre ai fatti, la KB formalizza le condizioni di compatibilità strutturale mediante regole logiche espressive (clausole di Horn):

```
% Elegibilità basata sul rispetto del budget massimo
    eligible_price(House, MaxBudget) :-
    house_price(House, Price),
    Price =< MaxBudget.

% Elegibilità basata sull'ubicazione geografica desiderata
    eligible_city(House, City) :-
    house_city(House, City).

% Raccomandazione finale determinata dalla composizione di più
  vincoli logici
    recommend_house(House, MaxBudget, City) :-
    house(House),
    eligible_price(House, MaxBudget),
    eligible_city(House, City).
```

Nel motore di inferenza Python, l'elegibilità è estesa con vincoli *hard* sulle caratteristiche accessorie richieste (es. presenza obbligatoria di ascensore o garage).

VALUTAZIONE

1. Esecuzione delle Interrogazioni e Spiegabilità

Attraverso lo script src/knowledge_base.py, è stata eseguita una query di test per un profilo cliente d'esempio avente i seguenti vincoli logici espliciti:

- **Budget massimo mensile:** 3500
- **Città di interesse:** taipei
- **Caratteristiche accessorie obbligatorie:** ['elevator'] (presenza ascensore)

Risultati dell'Inferenza Logica:

- Su una base di 100 immobili registrati nella KB, l'inferenza deterministica ha isolato esattamente **4 immobili idonei** che soddisfano contemporaneamente tutti i vincoli logici imposed.
- L'esito della query è stato esportato nel file results/kb_recommendations_sample.csv.

2. Complessità Computazionale del Ragionamento

Sia H il numero di immobili presenti nella KB (100) ed F il numero medio di caratteristiche per immobile $(2 \dots 4)$.

- La verifica della regola di elegibilità su un singolo immobile comporta un costo proporzionale al numero di feature F.
- La ricerca completa dell'insieme degli immobili raccomandati richiede di valutare tutti gli H immobili, per una complessità temporale complessiva pari a:

$$\mathcal{O}(H \times F)$$

Nel contesto considerato, tale complessità risulta estremamente contenuta e computazionalmente trascurabile, garantendo risposte in tempo reale (< 0.01 s).

3. Ruolo nel Sistema Ibrido

La Knowledge Base svolge un ruolo fondamentale e complementare rispetto ai modelli statistici di machine learning:

- Introduce filtri logici deterministici e vincoli espliciti che non dipendono da parametri statistici o appresi dai dati.
- Garantisce la completa **spiegabilità** del processo decisionale: se un immobile non viene raccomandato, è possibile isolare esattamente la regola o la clausola violata (es. prezzo oltre la soglia o assenza dell'ascensore).

SEZIONE ARGOMENTO 4: RISOLUZIONE DI UN CSP (CONSTRAINT SATISFACTION PROBLEM)

SOMMARIO

Nel contesto dell'Housing Matching, oltre alla classificazione binaria della singola coppia (cliente, immobile), è stato formalizzato e risolto un ulteriore problema decisionale complesso modellato come Constraint Satisfaction Problem (CSP).

L'obiettivo del modulo è costruire una **short-list ottimale di 10 immobili di lavoro/residenziali** che:

1. Siano pienamente compatibili con il profilo economico e geografico del cliente.
2. Rispettino vincoli strutturali e di qualità della selezione.
3. Massimizzino la varietà della proposta, evitando shortlist ridondanti.

Il problema presenta una spiccata natura combinatoria: dovendo selezionare 10 immobili distinti all'interno di un dataset di $N = 414$ elementi, lo spazio delle possibili configurazioni ha una cardinalità pari alla combinazione semplice:

$$\binom{414}{10} \approx 2.56 \times 10^{20}$$

Tale ampiezza dello spazio di ricerca rende inapplicabile l'esplorazione esaustiva e richiede l'adozione di tecniche di ricerca locale e metaeuristiche stocastiche.

STRUMENTI UTILIZZATI

Per la risoluzione del problema CSP sono stati implementati in Python due algoritmi stocastici di ricerca locale di letteratura:

- **Random Walk (Baseline):** Algoritmo di esplorazione casuale che seleziona uniformemente ad ogni passo una sotto-insieme di 10 immobili, calcola la relativa funzione di penalità e mantiene la migliore soluzione incontrata lungo le iterazioni.
- **Simulated Annealing (Metaeuristica):** Tecnica di ottimizzazione stocastica ispirata al processo di ricottura dei metalli. Partendo da una soluzione iniziale S, genera ad ogni passo una soluzione vicina S' modificando un singolo immobile nella shortlist. L'accettazione di una nuova soluzione avviene secondo la seguente regola:

- Se S' migliora la penalità ($\Delta E = E(S') - E(S) \leq 0$), viene sempre accettata.
- Se S' peggiora la penalità ($\Delta E > 0$), viene accettata con una probabilità variabile dipendente dalla temperatura corrente T:

$$P(\text{accettazione}) = \exp\left(-\frac{\Delta E}{T}\right)$$

- La temperatura diminuisce ad ogni iterazione mediante un fattore di raffreddamento geometrico $\alpha \in (0,1)$ ($T_{k+1} = \alpha \cdot T_k$), consentendo inizialmente di uscire da minimi locali e convergere progressivamente verso l'ottimo globale.

DECISIONI DI PROGETTO

1. Definizione dei Vincoli (Hard e Soft)

Il problema è stato formalizzato attraverso una combinazione bilanciata di vincoli rigidi e preferenze:

- **Vincoli Hard (Obbligatori):**
  1. *Dimensione Fissa:* La shortlist deve contenere esattamente 10 immobili distinti.
  2. *Prezzo Massimo:* Il prezzo unitario (unit_price) di ogni immobile selezionato non deve superare la soglia massima (45.0).
  3. *Distanza Massima dal Centro:* La distanza geografica (distance_km) non deve superare i 3.0 km dai trasporti/centro.
  4. *Budget Fit Medio:* Il valore medio di budget_fit della selezione deve essere ≥ 0.50.

- **Vincoli Soft (Preferenze di Diversità):** 5. *Diversità di Tipologia:* Non più di 4 immobili nella shortlist possono appartenere alla stessa tipologia (property_type). 6. *Diversità di Classe Energetica:* Non più di 4 immobili possono condividere la stessa classe energetica (energy_class).

2. Funzione di Penalità (Penalty Function)

Ogni soluzione candidata S viene valutata mediante una funzione di penalità cumulativa $f(S)$ che pondera le violazioni dei vincoli:

$$f(S) = 50 \cdot v_{\text{prezzo}} + 30 \cdot v_{\text{distanza}} + 100 \cdot \max\left(0{,}0.50 - \overline{\text{fit}}\right) + 15 \cdot v_{\text{tipo}} + 10 \cdot v_{\text{energia}}$$

Una penalità pari a 0.00 indica una soluzione ideale che soddisfa perfettamente tutti i vincoli *hard* e *soft*.

3. Configurazione degli Iperparametri e Protocollo Sperimentale

- *Simulated Annealing:* Temperatura iniziale $T_0 = 1000.0$, fattore di raffreddamento $\alpha = 0.95$, iterazioni massime per run $N_{\text{iter}} = 1000$.
- *Random Walk:* Iterazioni massime per run $N_{\text{iter}} = 1000$.
- *Benchmark:* Trattandosi di algoritmi stocastici, la valutazione è stata condotta eseguendo **10 run indipendenti** per ciascun metodo e calcolando il tempo medio di esecuzione e la penalità media ottenuta.

VALUTAZIONE

1. Risultati del Benchmark Sperimentale

Dall'esecuzione dello script src/csp_solver.py su 10 esecuzioni indipendenti sono stati ottenuti i seguenti risultati prestazionali:

Algoritmo	Tempo Medio di Esecuzione (s)	Penalità Media (f(S)f(S)f(S)) PDF	Penalità Migliore Trovata PDF
Random Walk (Baseline)	0.8257 s	6.07	0.00
Simulated Annealing	0.1321 s	0.00	0.00

2. Discussione dei Risultati

- **Simulated Annealing:** Ha dimostrato un'efficienza nettamente superiore, raggiungendo una **penalità media pari a** 0.00 in tutti i 10 run in soli 0.1321 secondi di media. La capacità dell'algoritmo di accertare mosse peggiorative alle alte temperature ha consentito di evitare minimi locali e convergere sistematicamente verso configurazioni ottimali.
- **Random Walk:** Pur riuscendo a trovare la soluzione a penalità zero in alcune run favorevoli, ha registrato una penalità media di 6.07 ed un tempo medio superiore (0.8257 s), confermando i limiti dell'esplorazione puramente casuale in uno spazio combinatorio ad alta dimensionalità.

La shortlist ottimale di 10 immobili priva di violazioni è stata esportata e salvata nel file results/csp_shortlist_optimal.csv

SEZIONE ARGOMENTO 5: RAGIONAMENTO SU CONOSCENZA INCERTA (RETE BAYESIANA E INFERENZA PROBABILISTICA)

SOMMARIO

Oltre all'approccio supervisionato predittivo e alla risoluzione del CSP combinatorio, nel sistema è stato implementato un modello probabilistico basato su una Rete Bayesiana discreta.

L'obiettivo di questo modulo è modellare esplicitamente la struttura di dipendenza causale e stocastica tra le variabili chiave del dominio immobiliare (budget_fit, distance_km, unit_price, house_age, property_type) per calcolare la probabilità posteriore:

$$P(\text{match} = 1 \mid \text{evidenze})$$

A differenza dei classificatori supervisionati classici, che restituiscono decisioni rigide o probabilità ottimizzate unicamente per la predizione pura, la rete bayesiana fornisce un framework di ragionamento in regime di incertezza pienamente interpretabile e trasparente. Essa consente di valutare la sensibilità delle decisioni al variare delle condizioni del cliente e di effettuare query probabilistiche anche in presenza di informazioni parziali.

La conoscenza incerta è rappresentata mediante una struttura di Grafi Diretti Aclicati (DAG) associata a Tabelle di Probabilità Condizionata (CPT - Conditional Probability Tables) stimate empiricamente dai dati.

STRUMENTI UTILIZZATI

Per l'implementazione del modulo di inferenza probabilistica sono stati impiegati i seguenti strumenti standard:

- **Pandas (pd.qcut):** Discretizzazione quantile-based delle variabili continue continuo-numeriche del dataset in intervalli ordinali a frequenza bilanciata.
- **Python / Bayesian Engine (src/bayesian_network.py):** Modulo dedicato alla discretizzazione del dataset housing_matching_supervised.csv, al calcolo data-driven delle CPT e all'esecuzione del motore di inferenza probabilistica per la stima delle probabilità posteriori dati scenari di evidenze.

DECISIONI DI PROGETTO

1. Discretizzazione delle Variabili Continue

Le reti bayesiane discrete richiedono la trasformazione delle feature numeriche continue in categorie ordinali discrete per rendere computazionalmente trattabile la stima delle Tabelle di Probabilità Condizionata (CPT):

- budget_fit_cat: partizionata in tre quantili (basso, medio, alto).
- distance_cat: partizionata in tre quantili (vicino, medio, lontano).
- price_cat: partizionata in tre quantili (economico, medio, costoso).
- age_cat: partizionata in tre quantili (nuova, media, datata).
- property_type: variabile categoriale originaria (apartment, villa, loft, studio).

2. Struttura Causa-Effetto del Grafo Orientato (DAG)

La struttura delle dipendenze condizionali è stata definita sulla base della conoscenza del dominio immobiliare:

- Le variabili budget_fit_cat, distance_cat e price_cat influenzano direttamente il nodo target binario match.
- L'assunzione di indipendenza condizionale consente di esprimere la distribuzione di probabilità congiunta del modello come prodotto delle distribuzioni condizionate:

$$P(\text{Match,Budget,Dist,Price}) = P(\text{Match} \mid \text{Budget,Dist,Price}) \cdot P(\text{Budget}) \cdot P(\text{Dist}) \cdot P(\text{Price})$$

3. Apprendimento Data-Driven dei Parametri (CPT)

Le Tabelle di Probabilità Condizionata (CPT) per $P(\text{Match} = 1 \mid \text{Budget,Dist,Price})$ sono state stimate direttamente dalle frequenze relative empiriche presenti nel dataset di 414 istanze. In caso di combinazioni di evidenze mai osservate nel campione di addestramento, il motore applica una probabilità a priori di fallback pari alla media globale del target $\overline{P(\text{match})}$.

VALUTAZIONE

1. Risultati dell'Inferenza Probabilistica su Scenari Decisionale

Attraverso lo script src/bayesian_network.py, l'inferenza probabilistica è stata valutata su tre differenti scenari di evidenze caratterizzati da condizioni operative differenti:

Scenario	Evidenze (budget_fit, distance, price)	Probabilità Posteriore P(Match=1|Evidenze)	Campioni Osservati nel Dataset
Scenario Ideale	Budget: alto, Distanza: vicino, Prezzo: costoso	**100.00%**	78
Scenario Flessibile	Budget: medio, Distanza: medio, Prezzo: medio	**58.49%**	53
Scenario Sfavorevole	Budget: basso, Distanza: lontano, Prezzo: costoso	**0.00%**	4

I risultati dettagliati dell'inferenza probabilistica sono stati salvati nel file results/bayesian_inference_results.csv.

2. Discussione dei Risultati e Relazione con i Modelli Supervisionati

- **Gradualità del Ragionamento:** A differenza del classificatore supervisionato (che fornisce una decisione binaria rigida 0 o 1), la Rete Bayesiana restituisce una misura esplicita dell'incertezza. Nello *Scenario Flessibile*, ad esempio, la probabilità del 58.49% esprime chiaramente una situazione di compatibilità moderata e sfumata.
- **Coerenza Causale:** Nello *Scenario Ideale* (alto budget fit e distanza ridotta dai trasporti) la probabilità di match raggiunge il 100.00%, mentre crolla allo 0.00% nello *Scenario Sfavorevole* (budget insufficiente, distanza elevata e prezzo costoso), confermando la perfetta rispondenza del modello probabilistico alle dinamiche del mercato immobiliare.
- **Trasparenza e Spiegabilità:** L'approccio probabilistico integra e arricchisce l'architettura ibrida del KBS: offre agli utenti finali (clienti o agenti immobiliari) una spiegazione interpretabile del perché una certa soluzione abitativa presenti una determinata probabilità di successo, evidenziando quali fattori aumentino o riducano la compatibilità complessiva.

CONCLUSIONI E RIFERIMENTI BIBLIOGRAFICI

CONCLUSIONI

Nel presente progetto è stato sviluppato un Knowledge-Based System (KBS) integrato per la risoluzione del problema dell'**Housing Matching**, affrontato unificando paradigmi di Intelligenza Artificiale complementari: apprendimento automatico predittivo, clustering non supervisionato, ragionamento simbolico esplicito, ottimizzazione vincolata e inferenza probabilistica.

I principali risultati ottenuti nei singoli moduli e la loro integrazione metodologica possono essere così sintetizzati:

1. **Apprendimento Non Supervisionato:** L'applicazione dell'algoritmo K-Means ha consentito di identificare 4 cluster latenti di mercato. L'analisi delle distanze euclidee dai centroidi ha permesso di isolare 21 anomalie geografico-economiche ($> 95°$ percentile), dimostrando la capacità del sistema di rilevare configurazioni abitative atipiche e rare.
2. **Apprendimento Supervisionato:** Il confronto condotto su un Model Zoo eterogeneo ha evidenziato le eccellenti prestazioni dei modelli ensemble ad albero. In particolare, **Gradient Boosting** e **Random Forest** hanno ottenuto un'area sotto la curva ROC pari a 1.0000 ed un'Accuracy fino al 100% sul test set separato, confermando la piena efficacia della Pipeline integrata e dell'ottimizzazione degli iperparametri via GridSearchCV con StratifiedKFold.
3. **Rappresentazione e Ragionamento Simbolico:** La Knowledge Base formalizzata in formato Prolog (housing_kb.pl) e replicata in Python ha garantito la verifica deterministica dei vincoli logici

(prezzo, ubicazione, servizi essenziali). Questo livello simbolico arricchisce il sistema fornendo totale spiegabilità ed interpretabilità del processo decisionale.

4. **Soddisfazione dei Vincoli (CSP):** La formalizzazione della generazione di una shortlist di 10 immobili come CSP ha dimostrato la netta superiorità del **Simulated Annealing** rispetto al Random Walk. L'algoritmo metaeuristico ha raggiunto sistematicamente una penalità media pari a 0.00 in soli 0.1321 s, risolvendo l'elevata complessità combinatoria dello spazio delle configurazioni.
5. **Ragionamento su Conoscenza Incerta:** La Rete Bayesiana discreta ha permesso di modellare le dipendenze condizionali del dominio immobiliare. Attraverso il calcolo della probabilità posteriore $P(\text{Match} = 1 \mid \text{Evidenze})$, il sistema offre una stima sfumata e graduale della compatibilità in contesti d'incertezza (es. 58.49% di probabilità per scenari con evidenze medie).

RIFERIMENTI BIBLIOGRAFICI

[1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Riferimento generale per agenti intelligenti, clausole di Horn, CSP e Reti Bayesiane).

[2] **Pedregosa, F., et al.** (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. (Riferimento per i moduli di preprocessing, K-Means, PCA e Model Zoo supervisionato).

[3] **Breiman, L.** (2001). *Random Forests*. Machine Learning, 45(1), 5-32. (Riferimento per l'algoritmo Random Forest).

[4] **Friedman, J. H.** (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 1189-1232. (Riferimento per il Gradient Boosting Classifier).

[5] **Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P.** (1983). *Optimization by Simulated Annealing*. Science, 220(4598), 671-680. (Riferimento per la metaeuristica Simulated Annealing nella risoluzione del CSP).

[6] **Pearl, J.** (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann. (Riferimento per la teoria delle Reti Bayesiane e l'inferenza probabilistica).

[7] **Clocksin, W. F., & Mellish, C. S.** (2003). *Programming in Prolog: Using the ISO Standard*. Springer Science & Business Media. (Riferimento per la formalizzazione dichiarativa della Knowledge Base).